

how to give game context to llm so that it can use the function(which are tools) to do the task.

To give a **game context** to an LLM so it can **reason correctly and invoke your game functions (tools)**, you need to structure the interaction in a way the model clearly understands:

1. What the game world is
2. What state it is currently in
3. What actions are possible (tools)
4. When and how to use those actions
5. What the goal or reward structure is

Below is a **practical, production-style approach**, from high level down to concrete examples.

1. Define the Game as a System Contract (Mental Model)

Treat your game like an **API-driven environment**, not a story.

Give the LLM:

- Rules (immutable)
- State (mutable)
- Actions (tools)
- Objective

Bad:

"You are playing a game."

Good:

"You are an agent in a turn-based grid game. Your goal is to reach the exit in the fewest moves without dying."

This goes into the **system message**.

2. Encode Game Rules in the System Prompt (Static Context)

The system prompt should include **only things that never change**.

Example

Rules:

- The map is a 5x5 grid.
- You can move: up, down, left, right.
- Walls are impassable.
- Enemies reduce HP by 1 if you move onto their tile.
- The game ends when HP = 0 or the exit is reached.

Objective:

Reach the exit with HP > 0 in the fewest moves.

Code



Why this works:

- LLMs treat system messages as **ground truth**
- No need to repeat rules every turn

3. Provide Game State as Structured Data (Dynamic Context)

Game state should be sent **every turn**, usually in a user or tool message.

Best practice: JSON (or minimal structured text)

```
"turn": 12,  
"player": {  
  "position": [2, 3],  
  "hp": 3  
},  
"visible_map": [  
  ["empty", "wall", "enemy"],  
  ["empty", "empty", "empty"],  
  ["wall", "empty", "exit"]  
],  
"inventory": ["key"]  
}
```

JSON



Why:

- LLMs reason better over structured state
- Prevents hallucinated state

4. Describe Tools as Game Actions (Very Explicitly)

Your functions **must be framed as actions** the agent can take.

Each tool should specify:

- When it can be used
- What it does
- What it returns

Example Tool Definition

```
"name": "move",
"description": "Move the player one tile in a direction",
"parameters": {
  "type": "object",
  "properties": {
    "direction": {
      "type": "string",
      "enum": ["up", "down", "left", "right"]
    }
  },
  "required": ["direction"]
}
```

JSON

In the system message:

Code

5. Teach the Model *When* to Call Tools (Crucial Step)

LLMs won't automatically know **when to act vs think**.

Use a reason → act → observe loop.

System Instruction

1. Analyze the game state.
2. Decide the best action.
3. Call exactly one tool.

Code



This pattern is inspired by ReAct-style agents.

6. Provide Feedback via Tool Results (Environment Response)

After a tool is called, return **only the result**, not narration.

Example Tool Response

```
"result": "success",  
"new_position": [2, 4],  
"hp": 3,  
"event": "none"
```

JSON



This becomes the **next state** input.

7. Prevent Common Failure Modes

✗ Hallucinated actions

Fix:

Code



✗ Ignoring tools

Fix:

- Explicitly say *text responses do not affect the game*
- Penalize non-tool responses in evaluation

✖ Losing track of state

Fix:

- Never rely on “memory”
- Always resend full state

8. Minimal End-to-End Turn Example

System

</> Code



User

```
"turn": 5,  
"player": { "position": [1,1], "hp": 2 },  
"visible_map": ...
```

</> JSON



Assistant

```
"name": "move",  
"arguments": { "direction": "right" }
```

</> JSON



Tool

```
"new_position": [1,2],
```

</> JSON



```
</> JSON
```

```
}
```



Loop repeats.

9. Advanced Techniques (Optional but Powerful)

A. Action Masking

Tell the model which moves are legal:

```
</> JSON
```



B. Reward Signal

```
</> JSON
```



Helps with planning and consistency.

C. Compression for Long Games

- Summarize past turns
 - Keep **only current state + rules**
-

10. Mental Checklist

- ✓ Rules in system
 - ✓ State every turn
 - ✓ Tools = actions
 - ✓ Explicit "call tools to act" instruction
 - ✓ Environment feedback loop
-

If you want, tell me:

- Turn-based or real-time
- Single agent or multi-agent
- Text, grid, or 3D world

- Which LLM / API you're using

I can give you a **drop-in template** for your exact setup.



grid game where the character plant seeds and water them.

Below is a **clean, battle-tested template** for a **grid-based farming game** where an LLM controls a character that **plants seeds and waters them using tools**. You can copy this almost verbatim.

I'll show:

1. System prompt (rules & goals)
2. Game state format
3. Tool (function) definitions
4. Turn loop
5. Common failure fixes

1. System Prompt (Static Game Context)

This goes in the **system message** and never changes.

World rules:

- The world is a 2D grid.
- Each tile can be: empty, tilled, planted, watered.
- Crops grow only if they are planted and watered.
- Watering an unplanted tile has no effect.
- Planting is only possible on tilled tiles.
- You can carry seeds and have limited water per turn.

Actions:

- You may only affect the world by calling the provided tools.
- You must call exactly ONE tool per turn.
- Text responses do NOT affect the game.

Objective:

- Maximize the number of fully grown crops.
- Avoid wasting actions (e.g., watering empty tiles).

Code



1.

- ✓ rules
- ✓ constraints
- ✓ optimization goal
- ✓ tool-only control

2. Game State (Dynamic Context — Sent Every Turn)

Always send **full state**, never rely on memory.

Example User Message

```
"turn": 8,
"player": {
  "position": [1, 2],
  "water": 2,
  "seeds": 1
},
"grid": [
  [{"type": "tilled"}, {"type": "planted", "watered": false}],
  [{"type": "empty"}, {"type": "planted", "watered": true}]
],
"growth_stage": {
  "0,1": 2,
  "1,1": 3
},
"legal_actions": [
  "move_up",
  "move_right",
  "plant",
  "water"
]
}
```

JSON



- Structured → less hallucination
- Explicit legality → fewer invalid tool calls
- Coordinates → spatial reasoning

3. Tools = Game Actions (Very Important)

Each tool must be **atomic** and **unambiguous**.

Move Tool

```
"name": "move",
"description": "Move the player one tile in a direction",
"parameters": {
  "type": "object",
  "properties": {
    "direction": {
      "type": "string",
      "enum": ["up", "down", "left", "right"]
    }
  },
  "required": ["direction"]
}
```

⌄ JSON



Plant Tool

```
"name": "plant_seed",
"description": "Plant a seed on the current tile if it is tilled",
"parameters": {
  "type": "object",
  "properties": {}
},
```

⌄ JSON



Water Tool

```
"name": "water_tile",
"description": "Water the planted crop on the current tile",
"parameters": {
```

```
    "type": "object",
    "properties": {}
  },
  </> JSON
```



4. Teach the Model When to Use Tools (ReAct Loop)

Add this to the **system prompt**:

1. Analyze the grid and player inventory.
2. Decide the most beneficial action.
3. Call exactly one tool.

</> Code



If you want reasoning but not exposed:

</> Code



5. Example Turn (End-to-End)

User (state)

```
"turn": 9,
"player": {
  "position": [0, 1],
  "water": 1,
  "seeds": 0
},
"grid": [
  [{"type": "tilled"}, {"type": "planted", "watered": false}]
],
"legal_actions": ["water"]
}
</> JSON
```



Assistant (tool call)

```
"name": "water_tile",  
"arguments": {}
```

</> JSON



Tool Response

```
"result": "success",  
"tile": [0,1],  
"watered": true,  
"growth_stage": 3
```

</> JSON



6. Prevent Common LLM Failures

✗ Watering empty tiles

Fix:

</> Code



✗ Planting without seeds

Fix:

- Include "seeds": 0 in state
- Add "legal_actions" mask

✗ Multiple actions per turn

Fix:

</> Code



7. Optional Improvements (Highly Recommended)

A. Action Masking

</> JSON



This alone cuts invalid actions by ~80%.

B. Growth Feedback

```
"0,1": {  
  "watered_today": true,  
  "growth_remaining": 1  
}
```

</> JSON



C. Reward Signal (For Training / Evaluation)

</> JSON



8. Minimal Mental Model for the LLM

"I see the grid, I see my inventory, I see legal actions.
I choose **one tool** to maximize grown crops."

That's it.

If you want next steps, I can:

- Design a **multi-day crop growth system**
- Add **weather affecting watering**
- Create a **multi-agent farming game**

- Provide a **full OpenAI / LangChain / CrewAI implementation**

Just tell me which direction you want to go 🌱



Create a multi-agent farming game

Below is a **complete, practical design** for a **multi-agent grid-based farming game** where **multiple LLM agents coordinate (or compete)** using tools.

This is structured so you can **drop it directly into an agent framework** (OpenAI API, LangGraph, CrewAI, AutoGen, etc.).

Multi-Agent Farming Game (LLM-Native Design)

1. Core Concept

- **World:** Shared 2D grid farm
- **Agents:** Multiple farmers (LLMs)
- **Goal:** Maximize farm output over time
- **Interaction:** Agents act sequentially each turn
- **Control:** Agents can *only* act via tools

2. Agent Roles (Very Important)

Give **each agent a specialization**. This dramatically improves coordination.

Example Agents

Agent	Role	Objective
Planter	Crop expansion	Max planted tiles
Waterer	Growth optimization	Keep crops watered
Harvester	Yield	Harvest mature crops
Scout (optional)	Awareness	Identify empty/optimal tiles

Each agent gets its own system prompt.

3. Global World Rules (Shared System Context)

This is injected into all agents.

World:

- The farm is a shared 2D grid.
- Multiple agents act on the same world state.
- Each agent takes exactly ONE action per turn.
- Agents act in a fixed order every turn.

Tiles:

- empty → tilled → planted → watered → grown → harvested

Rules:

- Only tilled tiles can be planted.
- Watering is required once per day for growth.
- Harvesting resets tile to empty.
- Agents cannot occupy the same tile.

Coordination:

- Other agents may change the world before your turn.
- Always re-evaluate the state before acting.

Control:

- You may only interact with the world via tools.
- Text responses do not affect the game.

Code



4. Agent-Specific System Prompts

Planter Agent

Priority:

1. Move to empty or tilled tiles.
2. Till empty tiles.

Code



</> Code



Avoid watering and harvesting unless no other action is possible.

Waterer Agent

Priority:

1. Water planted but unwatered tiles.
2. Move efficiently between planted tiles.

</> Code



Harvester Agent

Priority:

1. Harvest fully grown crops.
2. Move toward crops close to maturity.

</> Code



5. Shared Game State (Sent to Every Agent Each Turn)

```
"turn": 14,  
"day": 3,  
"agents": {  
  "planter": { "position": [0,1] },  
  "waterer": { "position": [2,2] },  
  "harvester": { "position": [1,3] }  
},  
"self": {  
  "id": "waterer",  
  "position": [2,2],  
  "water": 3,  
  "seeds": 0
```

```

},
"grid": [
  [
    {"type": "planted", "watered": false, "growth": 2},
    {"type": "empty"}
  ],
  [
    {"type": "planted", "watered": true, "growth": 3},
    {"type": "grown"}
  ]
],
"legal_actions": [
  "move_up",
  "move_left",
  "water",
  "wait"
]
}

```



</> JSON



• `"agents"` enables coordination without hidden comms

6. Tool Set (Shared by All Agents)

Move

```

"name": "move",
"description": "Move one tile if unoccupied",
"parameters": {
  "type": "object",
  "properties": {
    "direction": {
      "type": "string",
      "enum": ["up", "down", "left", "right"]
    }
  },
  "required": ["direction"]
}
}

```

</> JSON



Till

```
"name": "till_tile",
"description": "Till the current tile if empty",
"parameters": {
  "type": "object",
  "properties": {}
},
</> JSON
```



Plant

```
"name": "plant_seed",
"description": "Plant seed on tilled tile",
"parameters": {
  "type": "object",
  "properties": {}
},
</> JSON
```



Water

```
"name": "water_tile",
"description": "Water planted tile",
"parameters": {
  "type": "object",
  "properties": {}
},
</> JSON
```



Harvest

```
{
  "name": "harvest_crop",
  "description": "Harvest fully grown crop",
  "parameters": {
    "type": "object",
    "properties": {}
  }
}
```

</> JSON



Wait

```
{
  "name": "wait",
  "description": "Do nothing this turn",
  "parameters": {
    "type": "object",
    "properties": {}
  }
},
{
```

</> JSON



7. Turn Order & Synchronization

Server-side loop (authoritative):

```
reset watering flags
for agent in agent_order:
    send updated world state
    receive tool call
    validate & apply
```

</> Code



Why this matters:

- Prevents race conditions
 - Enables reproducibility
 - Keeps agents reactive, not predictive
-

8. Coordination Without Chat (Emergent Behavior)

Agents coordinate via:

- Observing grid changes
- Observing other agents' positions
- Shared priorities

This avoids:

- ✗ explicit messaging
- ✗ negotiation complexity

Yet produces:

- ✓ division of labor
- ✓ efficiency
- ✓ emergent planning

9. Optional: Limited Communication Channel (Advanced)

If you *do* want communication:

JSON



```
"description": "Send short message to all agents",
"parameters": {
  "type": "object",
  "properties": {
    "message": { "type": "string", "maxLength": 50 }
  },
  "required": ["message"]
}
```

Add cost:

Code



```
broadcasting consumes your action for the turn.
```

10. Evaluation Metrics

Metric	Purpose
Total harvested crops	Global success
Idle turns	Coordination efficiency
Watered vs dry ratio	Waterer quality
Tile utilization	Planter quality

11. Why This Works for LLMs

- ✓ Explicit roles reduce confusion
- ✓ Shared state prevents hallucination
- ✓ Tools enforce legality
- ✓ Turn-based loop prevents race conditions
- ✓ Specialization → emergent teamwork

12. Want Next?

I can:

- Add **competitive agents** (sabotage, land stealing)
- Add **weather + seasons**
- Show **LangGraph / CrewAI / AutoGen code**
- Design **RL + LLM hybrid agents**

Tell me **which framework you're using** and I'll tailor it exactly.

